Explained_alu — alu.v: The Arithmetic Logic Unit

Oct 12, 2026 · @shiv

1. Overview

`alu.v` is the processor's calculator. It takes two 8-bit numbers and a 3-bit operation code and produces the 8-bit result plus two flags: Z (result is zero) and C (carry, borrow or shifted-out bit). It is pure combinational logic, with no clock and no memory.

Item	Value
File	rtl/alu.v (100 lines)
Module name	alu
Pipeline stage	Stage 3 – Execute (EX)
Type of logic	Combinational
Calls (instantiates)	Nothing – leaf module
Called by	risc8_5stage_core.v, instance u_alu, lines 213–220
Receives alu_op from	control_unit.v (decoded in ID, carried through the ID/EX register)

1.1 Where it sits

```
tb_risc8_5stage.v
└── risc8_5stage_core.v
    ├── imem.v
    ├── control_unit.v   (makes alu_op)
    ├── regfile.v
    ├── hazard_unit.v
    ├── alu.v            <== THIS FILE
    └── dmem.v
```

1.2 Ports

Port	Dir.	Width	Meaning	Connected to (in risc8_5stage_core.v)
a	in	8	First operand	ex_opA (forwarding mux A)
b	in	8	Second operand	ex_alu_b (register value or immediate)
alu_op	in	3	Operation 0–7	idex_alu_op (from control_unit.v via ID/EX)
y	out	8	Result	ex_alu_y → EX/MEM register
c	out	1	Carry / borrow / shifted bit	ex_alu_c → flag_c
z	out	1	1 when y is 0	ex_alu_z → flag_z

1.3 Dependencies

Modules, functions or tasks called inside this file: none. It is a leaf module built only from Verilog operators.

Used by (file)	How	Instance	Lines
risc8_5stage_core.v	Module instance	u_alu	213–220

Must agree with: control_unit.v, which puts the same operation numbers (0–7) on alu_op.

1.4 Verilog constructs used

Construct	Meaning in plain words
`module` / `endmodule`	Start and end of a hardware block
`input` / `output` , `wire` / `reg`	Port direction; wire = connection, reg = assigned in an always block
`[7:0]`	Bus width: bits 7 down to 0
`localparam`	Named constant used only in this module

Construct	Meaning in plain words
`always @(*)`	Combinational block, recalculated when any input changes
`case` / `endcase` / `default`	Choose one action by value
`{x, y}`	Concatenation: join bits side by side
`assign`	Continuous connection of a wire to an expression

2. Code walkthrough, part by part

2.1 File header and timescale (lines 1–27)

The comment block records the file name, module, what it does, **who uses it** (risc8_5stage_core.v, Stage 3 EX) and **what it calls** (nothing). `` `timescale 1ns/1ps `` tells the simulator that 1 time unit = 1 ns; it does not affect hardware.

2.2 Module and ports (lines 29–36)

```
module alu (
    input  wire [7:0] a,        // Operand A : from forwarding mux ex_opA
(risc8_5stage_core.v)
    input  wire [7:0] b,        // Operand B : register value OR 8-bit
immediate
    input  wire [2:0] alu_op,   // Operation select : from control_unit.v
    output reg  [7:0] y,        // 8-bit result
    output reg        c,        // Carry-out (ADD), borrow (SUB), shifted-out
bit (SHL/SHR)
    output wire       z         // Zero flag : 1 when y == 0
);
```

`y` and `c` are `reg` only because they are assigned in an always block; no flip-flop is created. `z` is a `wire` because it is driven by `assign`.

2.3 Operation codes (lines 38–49)

```
    localparam ALU_ADD   = 3'd0,   // y = a + b
               ALU_SUB   = 3'd1,   // y = a - b
               ALU_AND   = 3'd2,   // y = a & b   (bitwise AND)
               ALU_OR    = 3'd3,   // y = a | b   (bitwise OR)
               ALU_XOR   = 3'd4,   // y = a ^ b   (bitwise XOR)
               ALU_SHL   = 3'd5,   // y = a << 1  (shift left by one)
               ALU_SHR   = 3'd6,   // y = a >> 1  (shift right by one)
               ALU_PASSB = 3'd7;   // y = b       (used by LDI: copy
immediate)
```

Link to another file: these numbers must be the same as the values control_unit.v puts on `alu_op` (for example ADD → 0, LDI → 7).

2.4 The always block and the carry default (lines 56–62)

```
    always @(*) begin
        c = 1'b0;
        case (alu_op)
```

`always @(*)` recalculates whenever any input changes, so synthesis builds gates. `c = 1'b0` gives the carry a default; without it, AND/OR/XOR/PASS would leave `c` unassigned and synthesis would create an unwanted **latch**.

2.5 ADD and SUB with carry (lines 63–70)

```
            ALU_ADD  : {c, y} = {1'b0, a} + {1'b0, b};
            ALU_SUB  : {c, y} = {1'b0, a} - {1'b0, b};
```

`{1'b0, a}` glues a 0 onto the left, making a 9-bit number. The 9-bit result is split: bit 8 → `c`, bits 7–0 → `y`. Example: 0xFF + 0x01 gives y = 0x00, c = 1 (overflow). For SUB, c = 1 means a **borrow** (first number was smaller).

2.6 Logic operations (lines 72–75)

```
            ALU_AND  : y = a & b;
            ALU_OR   : y = a | b;
            ALU_XOR  : y = a ^ b;
```

Bit by bit on all 8 bits. Carry stays 0 from the default.

2.7 Shifts (lines 77–83)

```
            ALU_SHL  : {c, y} = {a, 1'b0};
            ALU_SHR  : {y, c} = {1'b0, a};
```

SHL appends a 0 on the right (y = a × 2); the old bit 7 drops into `c`. SHR puts a 0 on the left (y = a ÷ 2); the old bit 0 drops into `c`.

2.8 Pass B and default (lines 85–91)

```
            ALU_PASSB: y = b;
            default  : y = 8'h00;
        endcase
    end
```

Operation 7 copies `b` to `y`. **LDI** uses it: the core puts the immediate on `b`. `default` is a safety net.

2.9 Zero flag (lines 93–100)

```
    assign z = (y == 8'h00);

endmodule
```

`z` is 1 when all result bits are 0. **Link:** risc8_5stage_core.v stores it in `flag_z`, which BZ and BNZ test.

3. Worked examples (a = 0x3C, b = 0x0F)

alu_op	Operation	y	c	z	Used by
0	a + b	0x4B	0	0	ADD, ADDI
1	a − b	0x2D	0	0	SUB
2	a AND b	0x0C	0	0	AND
3	a OR b	0x3F	0	0	OR
4	a XOR b	0x33	0	0	XOR
5	a << 1	0x78	0	0	SHL
6	a >> 1	0x1E	0	0	SHR

alu_op	Operation	y	c	z	Used by
7	b	0x0F	0	0	LDI

4. Common mistakes

- No default for `c`: synthesis creates a latch.
- Adding without the 9-bit extension: the carry is lost.
- Changing an alu_op number here without changing control_unit.v.

5. Viva questions

Q1. Why is y declared reg if there is no flip-flop? Anything assigned in an always block must be reg; with always @(*) and every path assigned, synthesis builds combinational logic.

Q2. How is the carry obtained? Both operands are extended to 9 bits; the 9th bit of the result is the carry.

Q3. What does C mean after SUB? C = 1 means a borrow: the first operand was smaller.

Q4. Why does LDI use the ALU? PASS B copies the immediate through, so LDI uses the same path as other ALU instructions.

Q5. Is the ALU sequential or combinational? Combinational: no clock, outputs follow inputs.